

QUEUE

Q6) Circular Linked List Implementation in Circular Queue

DS	Session	Queue	Question Information	Level 1	Challenge 56
Question description Lalitha is a B.Tech student. During her final year Campus Interview, she has an opportunity to get a job in a software company in Bangalore. The company provides Five months training period with Rs.30000/month Package. Then it will be incremented to Rs.55000 per month. At the end of the training, the examination was conducted for all freshers, Lalitha got a question paper and one of the questions comes under the concept of Queues in data structure that is Circular Linked List Implementation in Circular Queue. Can you help?					
Constraints First 5 elements inserted in array 0, next 5 elements inserted into array 1, remaining elements inserted into array 2.					

Input Format

First line indicates the number of elements to be inserted in the queues.

Second line indicates the elements.

Output Format

first line indicates the dequeue element of Q2

Second line indicates the dequeue element of Q1

Third line line indicates the dequeue element of Q0

Note: Refer sample input and output test cases

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
10
1 2 3 4 5 6 7 8 9 10
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 1
Deleted value = 2Elements in Circular Queue are:3 4
5 6 7 8 9 10
```

Test Case 2

INPUT (STDIN)

```
10
16 17 18 19 20 11 12 13 14 15
```

EXPECTED OUTPUT

```
Elements in Circular Queue are:16 17 18 19 20 11 12
13 14 15
Deleted value = 16
Deleted value = 17Elements in Circular Queue are:18
19 20 11 12 13 14 15
```

CODE:

```
C main.c > main
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node { int data; struct Node* next; };
5  typedef struct Queue { struct Node *front, *rear; } Queue;
6
7  void enqueue(Queue* q, int value) {
8      struct Node* t = (struct Node*)malloc(sizeof(struct Node));
9      t->data = value;
10     if (!q->front) q->front = t;
11     else q->rear->next = t;
12     (q->rear = t)->next = q->front;
13 }
14
15 int dequeue(Queue* q) {
16     if (!q->front) return -1;
17     int v = q->front->data;
18     if (q->front == q->rear) { free(ptr: q->front); q->front = q->rear = NULL; }
19     else { struct Node* t = q->front; q->rear->next = q->front = q->front->next; free(ptr: t); }
20     return v;
21 }
22
23 void displayQueue(struct Queue* q) {
24     struct Node* t = q->front;
25     printf(format: "Elements in Circular Queue are:");
26     if (t) do { printf(format: "%d", (t == q->front ? "" : " "), t->data); t = t->next; } while (t != q->front);
27 }
28
29 int main(void) {
30     int n, x;
31     if (scanf(format: "%d", &n) != 1) return 0;
32     Queue q = {NULL, NULL};
33     for (int i = 0; i < n && scanf(format: "%d", &x); i++) enqueue(q, &q, value: x);
34     displayQueue(q; &q);
35     printf(format: "\nDeleted value = %d\nDeleted value = %d", dequeue(q; &q), dequeue(q; &q));
36     displayQueue(q; &q);
37     printf(format: "\n");
38     return 0;
39 }
```

OUTPUT:

```
[suraj@suraj-83gs test]$ gcc main.c -o main
[suraj@suraj-83gs test]$ ./main
10
1 2 3 4 5 6 7 8 9 10
Elements in Circular Queue are:1 2 3 4 5 6 7 8 9 10
Deleted value = 2
Deleted value = 1Elements in Circular Queue are:3 4 5 6 7 8 9 10
[suraj@suraj-83gs test]$
```